WanderMate: Social Mobile Travel Planner

Task 2

Architecture Framework

IEEE 42010 Stakeholder Analysis & Nygard Architecture Decision Records

Contents

1 Introduction **2**

2 Stakeholder Identification (IEEE 42010) **3**

2.1 Identified Stakeholders . . . 3

2.2 Stakeholder Concerns . . . 4

2.3 Architecture Viewpoints . . . 5

2.3.1 VP1: Functional Viewpoint . . . 5

2.3.2 VP2: Information Viewpoint . . . 6

2.3.3 VP3: Deployment Viewpoint . . . 8

2.3.4 VP4: Development Viewpoint . . . 10

2.4 Stakeholder–Concern–Viewpoint Traceability . . . 11

3 Architecture Decision Records (ADRs) **12**

3.1 ADR-001: Use Firebase Realtime Database as an Event Bus for Collaboration (Not as a Primary Data Store) . . . 13

3.2 ADR-002: Adopt the API Gateway Pattern with Server-side Secret Proxying . . . 15

3.3 ADR-003: Implement the Facade + Strategy Pattern for Multi-provider POI Access . . . 17

3.4 ADR-004: Adopt an Offline-First Architecture with AsyncStorage and a Persistent Sync Queue . . . 21

4 ADR Summary and Cross-references **24**

5 Conclusion **25**

1. Introduction

WanderMate is a collaborative mobile travel planner built as a React Native (Expo) frontend communicating with a Node.js/Express backend, backed by MongoDB for persistent storage and Firebase Realtime Database for real-time event propagation. The system integrates multiple external APIs — Google Places, OpenRouteService (ORS), and a disabled Overpass adapter — all proxied through a backend API Gateway that acts as a secret vault and central policy enforcement point.

This document presents the **Architecture Framework** for WanderMate, structured in two parts:

1. **Stakeholder Identification (IEEE 42010)** — Identifying stakeholders, cataloguing their concerns, and defining viewpoints and views that address those concerns, following the ISO/IEC/IEEE 42010:2022 standard for architecture description.
2. **Architecture Decision Records (ADRs)** — Documenting four major design decisions using the Nygard ADR template, with rationale grounded in the WanderMate codebase.

> **Notation.** Throughout this document, requirement identifiers (NFR-06, FR-20) refer to the requirements catalogue in the Task 1 deliverable. Code references use `monospace` font and point to files in the WanderMate repository.

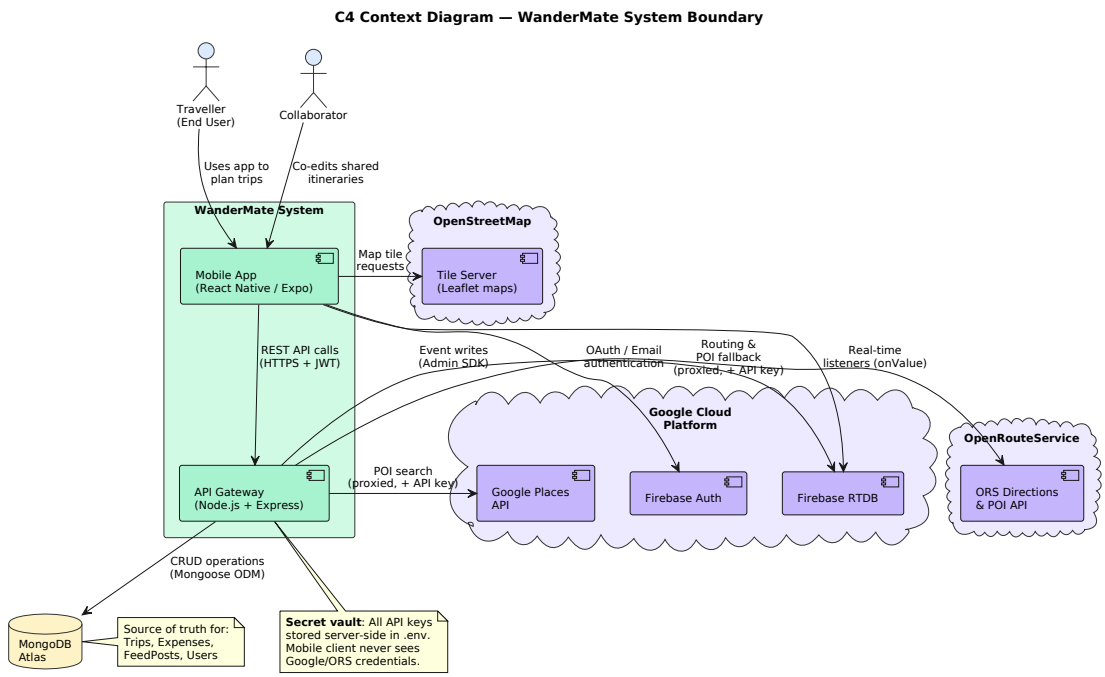

Figure 1: C4 Context Diagram: WanderMate system boundary showing the two internal components (Mobile App, API Gateway), five external systems (Google Places, ORS, Firebase Auth, Firebase RTDB, MongoDB Atlas), and two actor types (Traveller, Collaborator). All external API credentials reside exclusively on the backend.

2. Stakeholder Identification (IEEE 42010)

ISO/IEC/IEEE 42010:2022 provides a conceptual framework for architecture description, centered on four core concepts: **stakeholders** who have **concerns** about the system, which are addressed by **viewpoints** (conventions for constructing a view) that produce **views** (concrete architecture work-products). This section applies these concepts to the WanderMate system.

2.1 Identified Stakeholders

The following stakeholders have been identified for the WanderMate system. Each stakeholder classification is derived from their role in the system's lifecycle and their interests in the architecture.

#	Stakeholder	Description	Representative Roles
1	**End Users** (Travellers)	Individuals who use the mobile application to plan trips, discover POIs, manage budgets, and share itineraries.	Solo travellers, group trip organisers, travel enthusiasts browsing the social feed, collaborators on shared itineraries.
2	**Development Team**	Engineers responsible for building, testing, and evolving the WanderMate codebase across frontend and backend.	Frontend developers (React Native / Expo), backend developers (Node.js / Express), full-stack contributors.
3	**Project Supervisor / Evaluators**	Academic supervisors and evaluation panels who assess WanderMate's architectural quality, adherence to requirements, and software engineering practices.	Course instructor, project evaluators, external reviewers.
4	**System Administrator / DevOps**	Individuals responsible for deploying, configuring, monitoring, and maintaining the backend infrastructure and database services.	Backend deployer, MongoDB Atlas administrator, Firebase console operator, environment configuration manager.

#	Stakeholder	Description	Representative Roles
5	**External API Providers**	Third-party services whose APIs WanderMate consumes, each with usage quotas, SLAs, and terms of service that constrain the system's architecture.	Google Cloud (Places API), OpenRouteService, Firebase / Google Cloud Platform.

2.2 Stakeholder Concerns

Each stakeholder has specific concerns about the system. The table below exhaustively maps stakeholders to their primary concerns, with each concern traced to specific requirements from Task 1.

ID	Concern	Detailed Description	Stakeholders & Reqs
C1	**Usability & Responsiveness**	The mobile application must be intuitive, responsive, and provide quick access to core features (trip management, POI search, budgets, feed). UI interactions must feel native, with POI searches returning within 2 seconds and feed pages loading within 1.5 seconds.	End Users · NFR-01, NFR-02, NFR-13
C2	**Data Privacy & Security**	User credentials and session data must be protected via industry-standard authentication (Firebase JWT). Third-party API keys must never be exposed in the mobile app bundle. Per-trip access control must be enforced server-side.	End Users, Sys. Admin · NFR-10, NFR-11, NFR-12
C3	**Offline Availability**	Users must be able to view and edit itineraries while offline (e.g., in airports, remote areas, underground transport). Mutations must be queued and synchronised upon reconnection without data loss.	End Users · FR-03, NFR-07

ID	Concern	Detailed Description	Stakeholders & Reqs
C4	**Modifiability & Testability**	The codebase must be modular enough that adding or swapping an API provider (e.g., replacing Google Places with Mapbox) requires changes only in the relevant adapter, not in UI or routing layers. Each subsystem must be independently testable.	Dev Team, Supervisor · NFR-14, NFR-15
C5	**Deployment & Operations**	The system must be deployable with minimal configuration (environment variables in `.env`). Database connections, Firebase credentials, and API keys must be parameterised. The server must gracefully handle startup failures with clear error messages.	Sys. Admin · NFR-09
C6	**API Quota Compliance**	The system must operate within the free-tier quotas of all external APIs at launch: Google Places ($\leq$ 5,000 req/month), ORS ($\leq$ 2,000 req/day). Quota violations would incur financial cost and potentially break the service.	Supervisor, API Providers · NFR-06
C7	**Real-time Collaboration**	Multiple users editing the same trip must see each other's changes propagated within 500 ms. Conflict resolution must be deterministic and not require user intervention.	End Users · FR-20, FR-21, NFR-03

2.3 Architecture Viewpoints

Following IEEE 42010, we define four viewpoints. Each viewpoint provides conventions for constructing and interpreting a specific kind of architecture view that addresses a subset of the stakeholder concerns.

2.3.1 VP1: Functional Viewpoint

> **Purpose:** Describes the system's runtime functional elements, their responsibilities, interfaces, and the interactions between them.
>
> **Concerns addressed:** C1 (Usability & Responsiveness), C6 (Quota Compliance), C7

(Real-time Collaboration).

Stakeholders: End Users, API Providers, Development Team.

Model kinds: Component-and-connector diagrams, sequence diagrams.

View: The functional structure of WanderMate is decomposed into six subsystems and one cross-cutting layer, as documented in Task 1 Section 4 (Subsystem Overview):

1. **Mobile Client** (React Native / Expo) — all user interaction, Zustand state management (`tripStore`, `budgetStore`, `feedStore`, `authStore`, `activeTripStore`), Leaflet WebView map, and offline cache (`localCache`, `syncQueue`).
2. **API Gateway** (Node.js + Express) — single backend entry point with middleware stack: `helmet()` security headers, `authenticate()` Firebase JWT verification (`middleware/auth.js`), `generalLimiter` (100 req/15 min), and `cacheMiddleware` response cache (`middleware/cache.js`).
3. **POI & Routing Service** (`TravelService` Facade) — encapsulates `PlacesAdapter` (Google Places), `ORSAdapter` (OpenRouteService), and the disabled `OverpassAdapter` behind a unified interface with `node-cache` caching (30 min search, 24 hr details, 1 hr routes).
4. **Itinerary & Budget Engine** — `Trip` and `Expense` Mongoose models, trip/budget routes, `TripController` with owner/collaborator authorization checks, `TripObserver` event emitter for Firebase sync.
5. **Social Feed & Community Service** — `FeedPost` model with denormalised snapshots, feed pagination (`skip/limit`), publish/like/clone endpoints, follower graph (`User.followers[]`/`following[]`).
6. **Auth & User Profile** — Firebase Authentication (Email/Password + Google OAuth), `User` Mongoose model, `authStore` with `onAuthStateChanged` listener.
7. **Real-time Sync Infrastructure** (cross-cutting) — Firebase RTDB paths (`trips/{tripId}`, `budgets/{tripId}/lastUpdate`, `feed/{postId}`) with `onValue` listeners in frontend stores.

The runtime interactions follow five primary data flows: POI & Route Query, Collaborative Edit, Feed Publish, Itinerary Clone, and Offline Sync — each documented with step-by-step sequence descriptions in Task 1 Section 5.

2.3.2 VP2: Information Viewpoint

Purpose: Describes how information is stored, owned, managed, and accessed across the system. Defines the data ownership boundaries and consistency guarantees.

Concerns addressed: C2 (Data Privacy & Security), C3 (Offline Availability).

Stakeholders: End Users, System Administrator, Development Team.

Model kinds: Data model diagrams, data flow diagrams.

View: WanderMate uses a **dual-database architecture**: MongoDB as the authoritative store for all domain data, and Firebase RTDB as a thin event layer carrying notification payloads only.

MongoDB data model (Mongoose ODM):

- **Trip** (`models/Trip.js`) — nested document: `name`, `destination`, `startDate`, `endDate`, `owner` (Firebase UID), `collaborators[]` (UIDs), `days[]` → `stops[]` with `name`, `lat`,

lng, category, order, cost, etc. Indexes: (owner, createdAt), (collaborators), (isPublished, publishedAt).

- **Expense** (models/Expense.js) — separate collection, referenced by trip ObjectId. Fields: description, amount, category (accommodation | food | transport | activities | other), date, paidBy.
- **FeedPost** (models/FeedPost.js) — denormalised snapshot: trip (ref), author (UID), authorName, tripName, destination, duration, totalBudget, stopCount, likes[] (UIDs), likeCount. Indexes on createdAt, (author, createdAt), likeCount.
- **User** (models/User.js) — firebaseUid (unique), email, displayName, avatarUrl, followers[] (UIDs), following[] (UIDs), publishedTrips[] (ObjectId refs). Indexes on firebaseUid and email.

Firebase RTDB paths (thin event layer):

- trips/{tripId} — carries {updatedAt, lastEditedBy} on every mutation. Written by TripObserver event handler (services/tripObserver.js).
- budgets/{tripId}/lastUpdate — timestamp written on expense mutations.
- feed/{postId} — carries {author, tripName, createdAt} on publish.

Data ownership and consistency:

- MongoDB is the *single source of truth* for all complex queries (follower graphs, expense aggregations, feed pagination). Firebase is intentionally kept thin — it does not store full documents.
- The TripObserver (services/tripObserver.js) uses Node.js EventEmitter to decouple business logic from Firebase writes. The TripController emits events (TRIP_CREATED, TRIP_UPDATED, TRIP_DELETED, COLLABORATORS_UPDATED); the observer handles Firebase propagation.
- Client-side localCache (AsyncStorage) provides offline reads with infinite TTL; the syncQueue queues write operations for replay on reconnect.

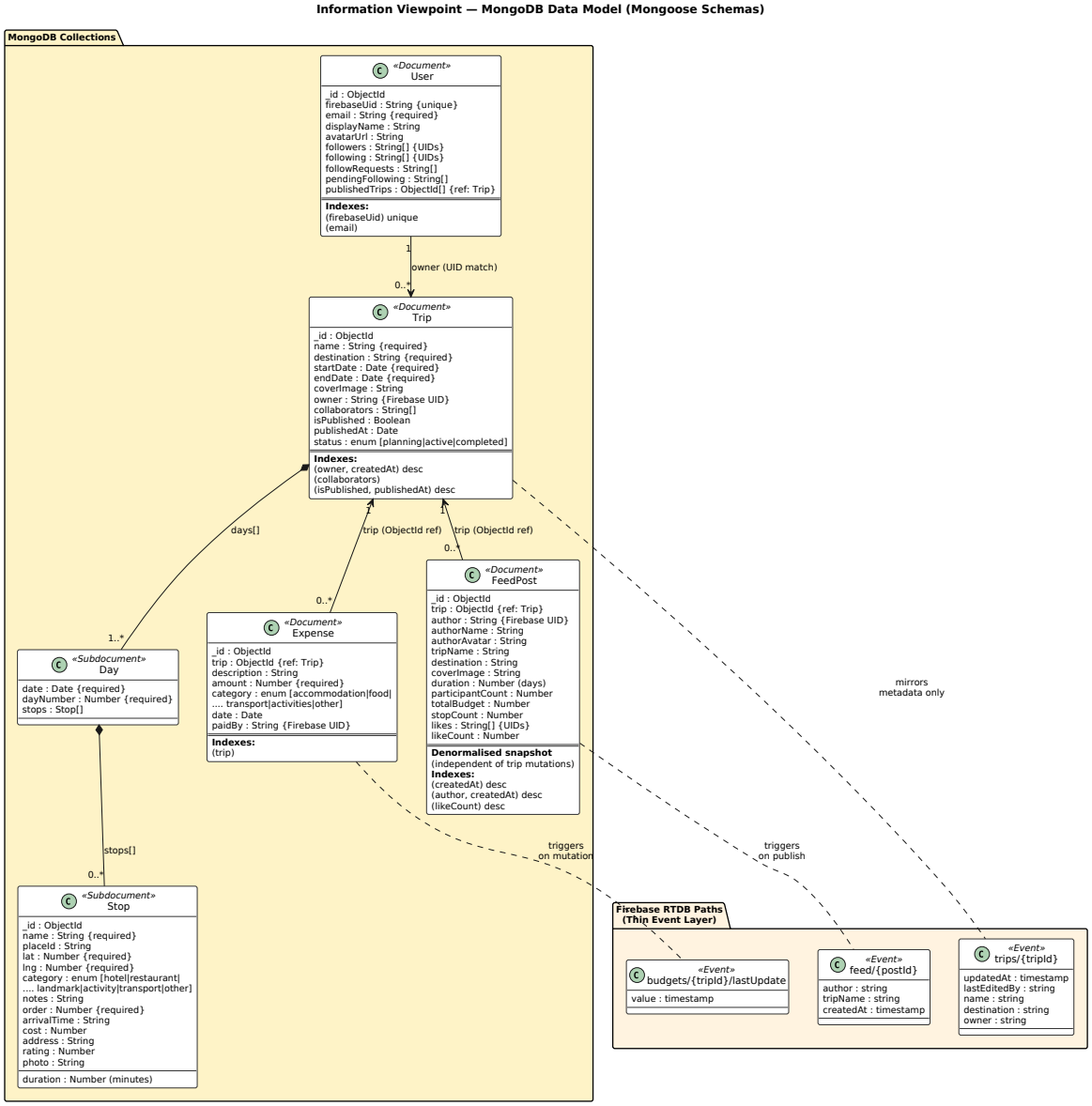

Figure 2: Information Viewpoint — MongoDB data model (Mongoose schemas) and Firebase RTDB event paths. The four MongoDB collections (Trip, Expense, FeedPost, User) serve as the authoritative data store; the three Firebase paths carry lightweight event notifications only.

2.3.3 VP3: Deployment Viewpoint

> **Purpose:** Describes the mapping of software artifacts to the physical and cloud infrastructure, including network topology, service dependencies, and configuration management.
>
> **Concerns addressed:** C5 (Deployment & Operations).
>
> **Stakeholders:** System Administrator, Development Team.
>
> **Model kinds:** Deployment diagrams, infrastructure topology.

View: WanderMate is deployed as a two-tier architecture with three managed cloud services:

Component	Runtime Target	Configuration & Notes
Mobile Client	Android device / Expo Go	Built with Expo SDK 54 (managed workflow). Uses Expo Router for file-based navigation. API base URL configured via `frontend/config/env.ts` (`API_BASE_URL`).
API Gateway	Node.js process (single instance)	Entry point: `backend/src/index.js`. Listens on port 5000. All secrets loaded from `backend/.env`: `MONGODB_URI`, `FIREBASE_*` credentials, `GOOGLE_PLACES_API_KEY`, `ORS_API_KEY`. Dev mode: `node -watch src/index.js`.
MongoDB	MongoDB Atlas (cloud)	Connection via `MONGODB_URI` in `.env`. Configured in `config/database.js` with Mongoose ODM (v9.4). Indexes pre-defined in model schemas for query performance.
Firebase RTDB	Google Cloud (managed)	Configured via `FIREBASE_DATABASE_URL`. Backend uses Firebase Admin SDK (singleton in `config/firebase.js`). Frontend uses Firebase JS SDK (singleton in `config/firebase.ts`) with AsyncStorage auth persistence.
Firebase Auth	Google Cloud (managed)	Providers: Email/Password, Google Sign-In (OAuth 2.0 via `expo-auth-session`). Web client ID configured in `frontend/config/env.ts`.

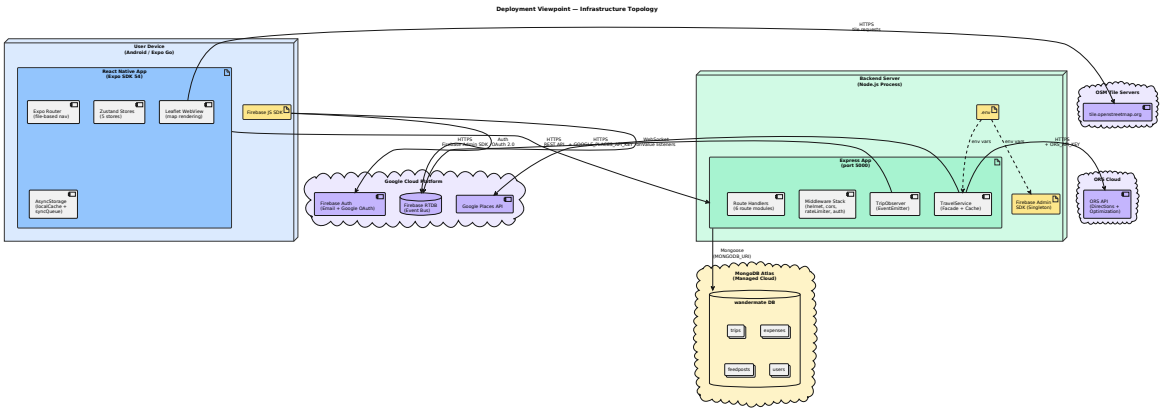

Figure 3: Deployment Viewpoint — infrastructure topology showing the User Device (React Native app with AsyncStorage), Backend Server (Express + middleware + TravelService), MongoDB Atlas, Firebase services, and external API providers. All connections labelled with protocol and credential flow.

2.3.4 VP4: Development Viewpoint

> **Purpose:** Describes the organisation of the codebase into modules, packages, and layers to support the team's development, testing, and evolution workflow.
>
> **Concerns addressed:** C4 (Modifiability & Testability).
>
> **Stakeholders:** Development Team, Project Supervisor.
>
> **Model kinds:** Package diagrams, layer diagrams, module decomposition.

View: The WanderMate codebase follows a **four-layer architecture** on both frontend and backend, with strict downward dependency flow:

Backend layers:

1. **Presentation** — Express route handlers (`routes/trips.js`, `routes/poi.js`, `routes/budget.js`, `routes/feed.js`, `routes/users.js`, `routes/routes.js`) and the `TripController` class.
2. **Business Logic** — `TravelService` Facade (`services/travelService.js`), `TripObserver` EventEmitter (`services/tripObserver.js`).
3. **Data Access** — Mongoose models (`models/Trip.js`, `models/Expense.js`, `models/FeedPost.js`, `models/User.js`).
4. **Infrastructure** — Config files (`config/database.js`, `config/firebase.js`), middleware (`middleware/auth.js`, `middleware/cache.js`, `middleware/rateLimiter.js`).

Frontend layers:

1. **View** — Expo Router screens (`app/(tabs)/`, `app/(auth)/`, `app/trip/`), reusable components (`components/`).
2. **State Management** — Zustand stores (`stores/tripStore.ts`, `stores/budgetStore.ts`, `stores/feedStore.ts`, `stores/authStore.ts`, `stores/activeTripStore.ts`).
3. **Service** — API client (`services/api.ts` with Axios interceptor for JWT injection), sync queue and local cache (`services/syncQueue.ts`).
4. **Infrastructure** — Firebase SDK init (`config/firebase.ts`), environment config (`config/env.ts`).

This layered structure ensures that replacing a data provider (e.g., swapping Google Places for Mapbox) requires changes only in layer 2 (a new adapter in `services/`) and the `TravelService` constructor — no changes to routes, stores, or UI screens. This was empirically validated when the `OverpassAdapter` was disabled: the change was confined to commenting out the import in `travelService.js` (NFR-15 compliance).

2.4 Stakeholder–Concern–Viewpoint Traceability

The following matrix provides full traceability from stakeholders through concerns to the viewpoints that address them:

Stakeholder	Primary Concerns	Addressed by Viewpoints	NFRs Traced
End Users	C1 (Usability), C3 (Offline), C7 (Real-time Collab.)	VP1 (Functional), VP2 (Information)	NFR-01, NFR-07, NFR-03
Development Team	C4 (Modifiability & Testability)	VP4 (Development), VP1 (Functional)	NFR-14, NFR-15
Project Supervisor	C4 (Modifiability), C6 (Quota Compliance)	VP4 (Development), VP1 (Functional)	NFR-06, NFR-14
System Administrator	C2 (Security), C5 (Deployment)	VP2 (Information), VP3 (Deployment)	NFR-10, NFR-11
API Providers	C6 (Quota Compliance)	VP1 (Functional)	NFR-06

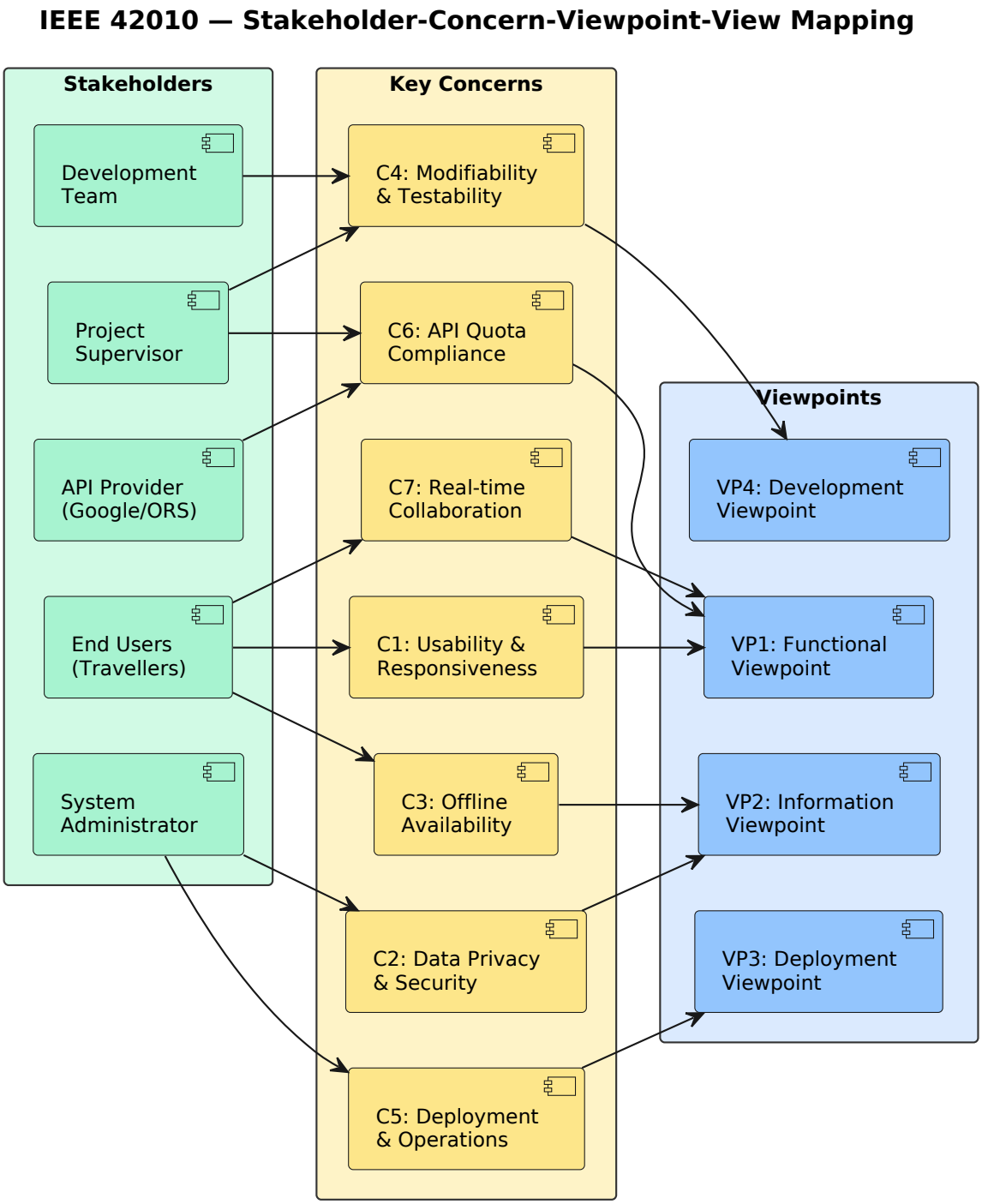

Figure 4: IEEE 42010 traceability: stakeholders → concerns → viewpoints. Each arrow indicates that a concern raised by a stakeholder is addressed by the corresponding viewpoint.

3. Architecture Decision Records (ADRs)

The following Architecture Decision Records document major design decisions using the **Nygard ADR template** (Title, Status, Context, Decision, Consequences). Each

ADR is grounded in specific code artefacts and requirements from the WanderMate repository.

3.1 ADR-001: Use Firebase Realtime Database as an Event Bus for Collaboration (Not as a Primary Data Store)

ADR-001: Use Firebase RTDB as a Thin Event Layer

Status: **Accepted**

Context

WanderMate requires real-time collaborative editing: when one user modifies a shared itinerary, all connected collaborators must see the change within 500 ms (NFR-03). At the same time, the system needs to support complex queries that Firebase RTDB is not optimised for:

- Paginated feed queries sorted by `createdAt` with follower-based filtering (`FeedPost.find({author: {$in: following}}).sort({createdAt: -1}).skip(skip).limit(limit)` in `routes/feed.js`, line 24).
- Per-trip expense aggregations with category breakdowns (server-side budget summary computation in `routes/budget.js`).
- Follower graph queries using adjacency-list relationships (`User.followers[]`, `User.following[]`).
- Cross-collection joins between `Trip`, `Expense`, and `User` documents during itinerary clone (`routes/feed.js`, lines 148–208).

We evaluated two architectural approaches:

1. **Firebase-primary:** Use Firebase RTDB as the primary data store for all trip data, with real-time listeners providing both persistence and collaboration.
2. **MongoDB-primary + Firebase-events:** Use MongoDB as the authoritative store for domain data, with Firebase serving only as a notification layer to trigger client re-fetches.

Decision

We will use **MongoDB as the single source of truth** for all domain data and **Firebase RTDB as a thin event bus** that carries only notification payloads (timestamps and metadata), not full documents.

Concretely:

- Backend route handlers persist all mutations to MongoDB via Mongoose models, then emit events on the `TripObserver` EventEmitter (`services/tripObserver.js`).
- The `TripObserver` writes lightweight payloads to Firebase paths:
  - `trips/{tripId}` ← `{name, destination, owner, updatedAt: Date.now()}`
  - `budgets/{tripId}/lastUpdate` ← timestamp
  - `feed/{postId}` ← `{author, tripName, createdAt: Date.now()}`
- Frontend stores subscribe via `onValue` listeners (`tripStore.subscribeTripUpdates()`, `budgetStore.subscribeBudgetUpdates()`) and, upon receiving a notification, re-fetch the full authoritative data from MongoDB via the API.

Consequences

Positive:

- MongoDB remains the single source of truth, avoiding dual-write consistency issues.
- Complex queries (pagination, aggregations, graph traversals) are handled by MongoDB's query planner with proper indexes, not by Firebase's limited query model.
- Firebase costs scale only with notification volume, not with data storage.
- The `TripObserver` EventEmitter (`services/tripObserver.js`) decouples business logic from sync infrastructure — route handlers call `tripEvents.emit('TRIP_UPDATED', trip._id, req.user.uid)` (e.g., `tripController.js`, line 104) without knowing about Firebase at all.

Negative:

- The re-fetch pattern (notification → API call → MongoDB read) adds one HTTP round-trip compared to a Firebase-primary approach where data would be pushed directly.
- Two infrastructure dependencies (MongoDB + Firebase) instead of one; both must be operational for full functionality.
- Conflict resolution relies on last-write-wins timestamps (`updatedAt`) rather than Firebase's built-in conflict resolution; complex concurrent edits may lose intermediate states.

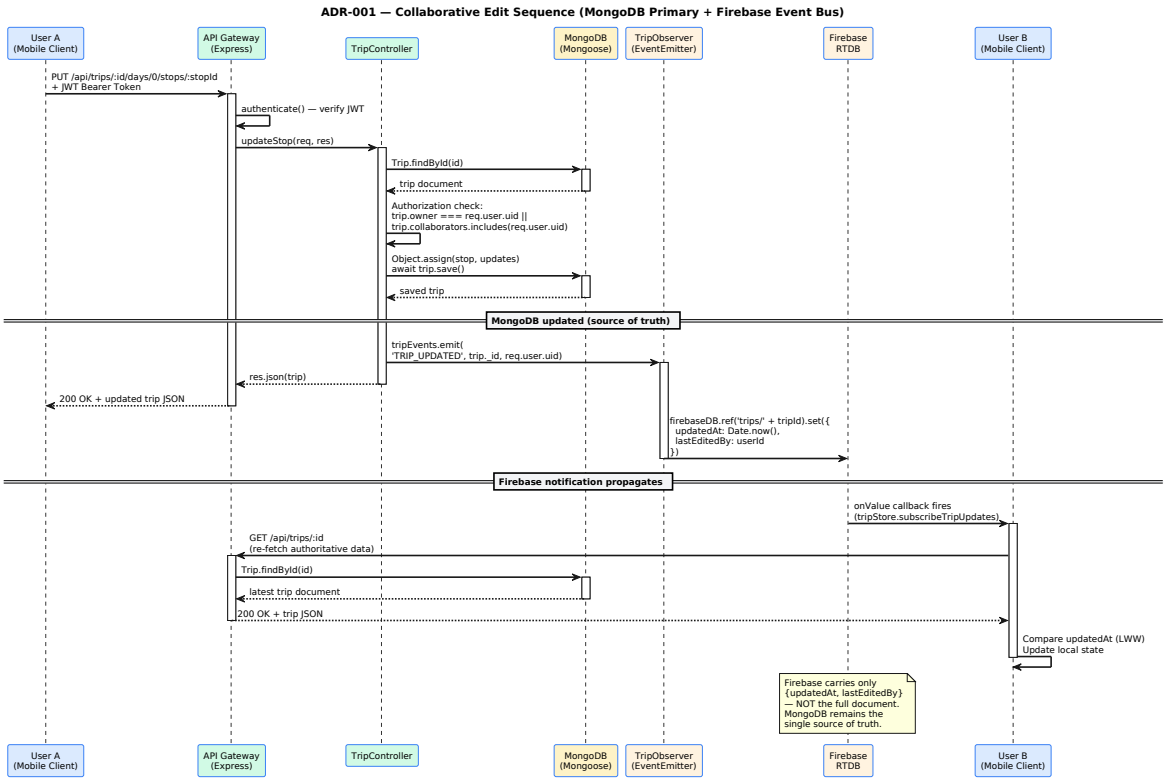

Figure 5: ADR-001 Sequence Diagram: collaborative edit flow. User A mutates a stop via the API Gateway, which persists to MongoDB (source of truth), then emits a `TRIP_UPDATED` event via `TripObserver`. The observer writes a lightweight notification to Firebase RTDB. User B's `tripStore` receives the `onValue` callback and re-fetches the authoritative trip from MongoDB.

3.2 ADR-002: Adopt the API Gateway Pattern with Server-side Secret Proxying

ADR-002: Backend API Gateway as Secret Vault and Policy Enforcement Point

Status: **Accepted**

Context

WanderMate consumes three external APIs with sensitive credentials:

- **Google Places API** — requires a `GOOGLE_PLACES_API_KEY` with billing enabled. If extracted from a mobile bundle, it could be used by attackers to make arbitrary calls, incurring cost.
- **OpenRouteService API** — requires an `ORS_API_KEY` with a daily quota of 2,000 requests.
- **Firebase Admin SDK** — requires a private key (`FIREBASE_PRIVATE_KEY`) that grants unrestricted access to the Firebase project (database administration, user management, custom token generation).

React Native applications are compiled into JavaScript bundles that can be extracted and decompiled from APK/IPA files using tools like `apktool` or `jadx`. Any API key embedded in the client-side bundle is, for practical purposes, a public key.

NFR-11 explicitly requires: "Third-party API keys shall never be embedded in the mobile app bundle."

Two approaches were considered:

1. **Direct client-to-API calls:** The mobile client calls external APIs directly, embedding keys in the bundle or fetching them at runtime from a config endpoint.
2. **Backend proxy:** A server-side API Gateway proxies all external API calls, storing keys as environment variables that never leave the server process.

Decision

We will implement a **Node.js/Express API Gateway** (`backend/src/index.js`) that:

1. Loads all API keys from `.env` via `dotenv` (line 1: `require('dotenv').config()`).
2. Applies a middleware stack on every request:
   - `helmet()` — sets security headers (`X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security`, etc.).
   - `cors({origin: '*'})` — allows cross-origin requests from the Expo development server.
   - `generalLimiter` — 100 requests per 15 minutes per IP (`middleware/rateLimiter.js`, line 6).
   - `authenticate` — verifies the Firebase JWT bearer token via `admin.auth().verifyIdToken()` (`middleware/auth.js`, lines 8–23).
3. Proxies external API calls exclusively through the `TravelService` Facade, which instantiates adapters with keys from `process.env`:
   - `this.places = new PlacesAdapter(process.env.GOOGLE_PLACES_API_KEY)` (`travelService.js`, line 11).
   - `this.ors = new ORSAdapter(process.env.ORS_API_KEY)` (`travelService.js`, line 12).

The mobile client's `frontend/services/api.ts` configures a single Axios instance with

`baseURL: API_BASE_URL` (pointing to the gateway) and injects the user's Firebase ID token via a request interceptor:

```
api.interceptors.request.use(async (config) => {
    const { auth } = await import('../config/firebase');
    const token = await auth.currentUser?.getIdToken();
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
});
```

The client has no knowledge of Google Places keys, ORS keys, or Firebase Admin credentials.

Consequences

Positive:

- **Secret protection:** API keys exist only in server memory, loaded from `.env`. Zero secrets in the mobile bundle (NFR-11 satisfied).
- **Centralized policy enforcement:** Authentication, rate limiting, response caching, and error handling are applied uniformly in one middleware stack, not duplicated across clients.
- **Quota control:** The gateway's `externalApiLimiter` (`rateLimiter.js`, line 14: 50 requests/hour/IP) prevents any single user from exhausting free-tier quotas (NFR-06).
- **Unified error format:** All errors are caught and returned as structured JSON (`{error: "message"}`) with appropriate HTTP status codes (NFR-09).

Negative:

- All external API traffic traverses the gateway, making it a single point of failure. If the Node.js process crashes, no POI search or routing is available.
- Added network latency: each request incurs a client → gateway → external API round trip instead of a direct client → API call.
- Operational overhead of maintaining and deploying a backend service (environment configuration, monitoring, scaling).

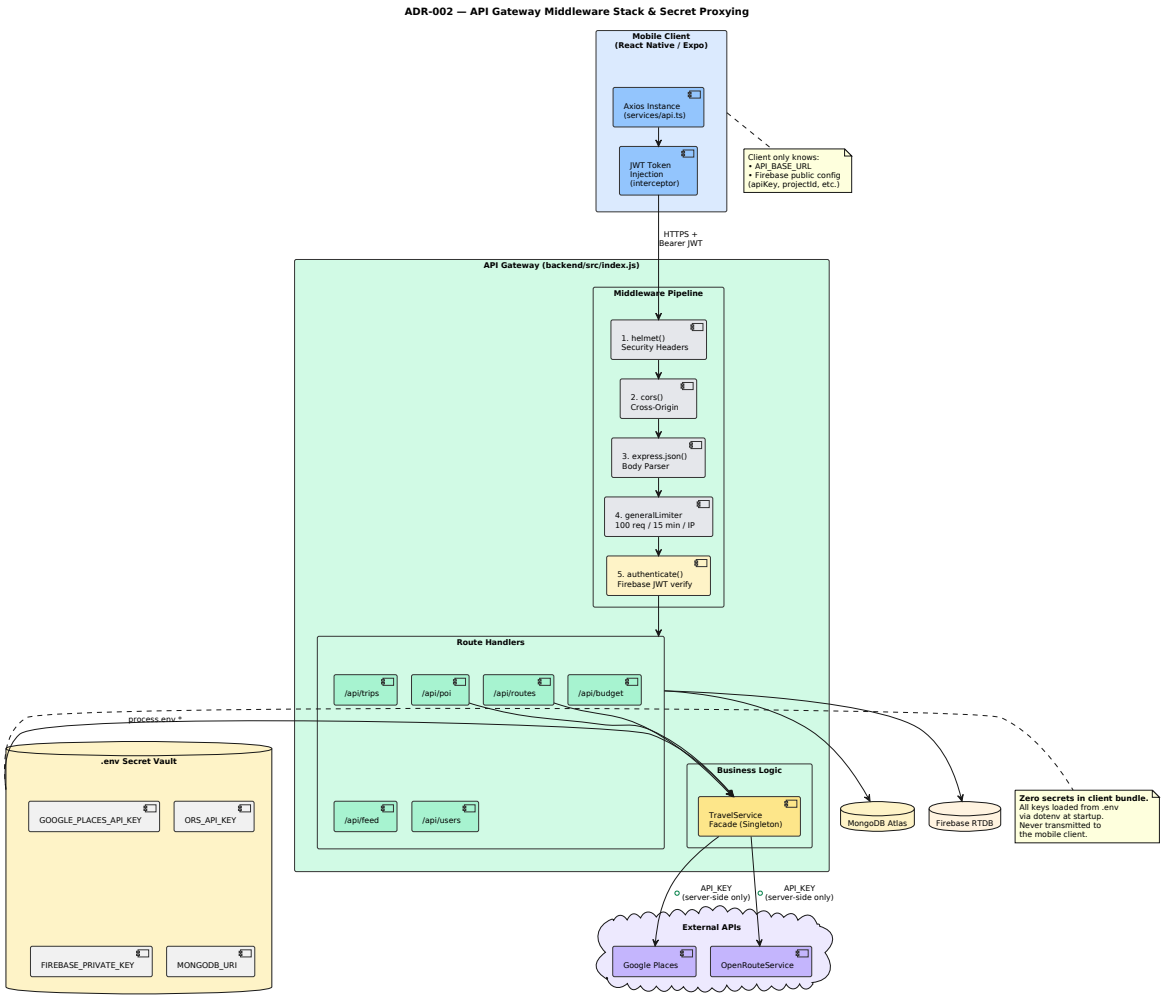

Figure 6: ADR-002 Component Diagram: API Gateway middleware pipeline and secret proxying flow. Requests from the mobile client pass through five middleware stages (helmet → cors → body parser → rate limiter → JWT auth) before reaching route handlers. External API keys are loaded from `.env` and injected by the TravelService Facade — the mobile client never sees them.

3.3 ADR-003: Implement the Facade + Strategy Pattern for Multi-provider POI Access

ADR-003: TravelService Facade with Interchangeable Adapter Strategies

Status: Accepted

Context

WanderMate's POI discovery and routing features require integration with multiple external APIs, each with different protocols, response formats, quota limits, and failure modes:

Provider	Protocol	Free-tier Quota	Data Quality
Google Places	REST, JSON	5,000 req/month	Rich (photos, ratings, hours, price levels)
ORS (POI)	REST, GeoJSON	2,000 req/day	Basic (name, category, coordinates)
ORS (Routing)	REST, GeoJSON	2,000 req/day	Good (directions + VROOM optimization)
Overpass	Overpass QL	Unlimited (mirrors)	Varies (raw OSM data)

Two architectural approaches were considered:

1. **Direct API calls in routes:** Each Express route handler directly calls whichever provider it needs, with inline response normalisation, caching, and fallback logic.
2. **Facade + Strategy:** A single `TravelService` class encapsulates all provider logic behind a unified interface, delegating to interchangeable adapter classes.

Without a unifying layer, adding or swapping a provider would require modifying every route handler that uses POI data — violating NFR-15 (provider swappability requires changes only in the adapter and Facade).

Decision

We will implement a `TravelService` class (`services/travelService.js`) that combines the **Facade** and **Strategy** patterns:

1. **Facade interface:** Five public methods — `searchPOI()`, `searchByCategory()`, `getPlaceDetails()`, `getRoute()`, `optimizeRoute()` — abstract away all provider and caching logic.
2. **Strategy adapters:** Three adapter classes, each wrapping a specific API behind a common normalised output:
   - `PlacesAdapter` (`services/placesAdapter.js`) — 168 lines, wraps Google Places (Text Search, Nearby Search, Find Place, Place Details). Normalises responses via `_normalize(place)` with category detection (`_detectCategory(types)`).
   - `ORSAdapter` (`services/orsAdapter.js`) — 240 lines, wraps ORS POI and routing endpoints. Normalises POIs via `_normalizePOI(feature)`. Implements both simple routing (`/v2/directions`) and multi-stop optimization (`/optimization`).
   - `OverpassAdapter` (`services/overpassAdapter.js`) — 115 lines, currently disabled. Uses mirror failover across multiple Overpass servers.
3. **Singleton:** A module-level instance (`let travelServiceInstance = null`) with a `getTravelService()` factory function ensures one instance is shared across all route modules.
4. **Caching:** A single `node-cache` instance inside the Facade applies per-method TTLs: 30 minutes for searches (`poi_search_*`), 24 hours for details (`poi_details_*`), 1 hour for routes (`route_*`).
5. **Fallback chain:** `searchPOI()` attempts Google Places first, catches failures, and falls through to ORS:

```
results = await this.places.searchPOI(query, lat, lng, radius)
                .catch(err => { return []; });
if (results.length === 0) {
    results = await this.ors.searchPOI(query, lat, lng, radius)
                  .catch(err => { return []; });
}
```

Consequences

Positive:

- **Provider independence:** Route handlers in `routes/poi.js` call `travelService.searchPOI()` (line 18) without knowing which provider responded. Adding Mapbox or HERE requires only a new adapter class and a

registration line in the `TravelService` constructor.

- **Validated swappability:** The `OverpassAdapter` was disabled by commenting out its import in `travelService.js` — zero changes to routes, stores, or UI components. This validates NFR-15.
- **Centralized caching:** One `node-cache` instance serves all providers; no duplicate cache logic. Cache key generation uses method name + parameters (`poi_search_${query}_${lat}_${lng}`).
- **Graceful degradation:** The fallback chain (Google → ORS → empty) is invisible to consumers, satisfying NFR-08.

Negative:

- An additional layer of abstraction increases code navigation complexity; developers must trace calls through the Facade to reach adapter implementations.
- The normalised POI interface is a least-common-denominator: rich Google Places data (photos, opening hours, price levels) is flattened alongside ORS's basic data, potentially underutilising provider-specific features.
- The fallback chain introduces latency when the primary provider fails. Two sequential API calls (Google → ORS) are made before returning results, though caching mitigates this for repeated queries.

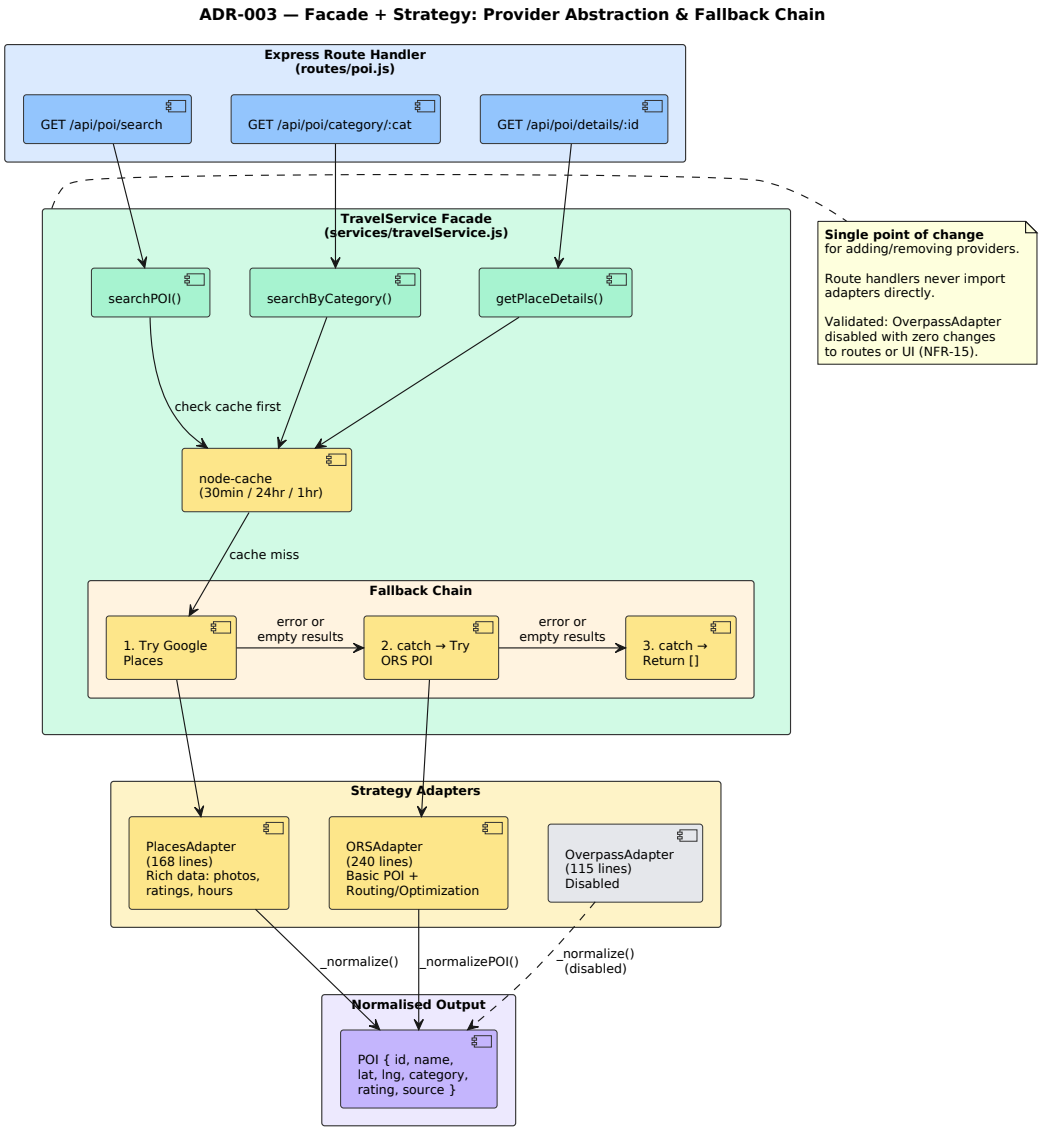

Figure 7: ADR-003 Component Diagram: the TravelService Facade receives calls from route handlers, checks the `node-cache`, and dispatches to the fallback chain (Google Places → ORS → empty array). All adapters normalise responses to a common POI interface. The OverpassAdapter is implemented but currently disabled.

3.4 ADR-004: Adopt an Offline-First Architecture with AsyncStorage and a Persistent Sync Queue

ADR-004: Offline-First Client with AsyncStorage Cache and SyncQueue

Status: Accepted

Context

Travel applications must function in environments with unreliable or no connectivity: airports during boarding, remote hiking trails, underground metro stations, and international roaming situations where data is expensive. WanderMate users must be able to:

- View their itineraries, trip details, and budget data while offline (FR-03, NFR-07).
- Edit trips (add/remove stops, modify notes) while offline, with changes synchronised when connectivity is restored.
- Not lose any data due to transient network failures.

Two approaches were considered:

1. **Online-required:** All operations require network connectivity. Offline access shows a "No connection" error screen.
2. **Offline-first:** All reads are served from a local cache first. Writes are queued locally and replayed to the server on reconnect.

Decision

We will implement an **offline-first architecture** using two modules in `frontend/services/syncQueue.ts`:

1. `localCache` — a key-value cache backed by AsyncStorage with configurable TTL:

   ```
   export const localCache = {
     async get(key, maxAge = 3600000) { /* 1 hour default */ },
     async set(key, data) { /* stores {data, timestamp} */ },
     async clear(key) { /* removes entry */ }
   };
   ```

   Every Zustand store method (e.g., `tripStore.fetchTrips()`) follows a try/catch pattern: attempt the API call, and on failure, read from `localCache` with infinite TTL (`Infinity`):

   ```
   try { response = await api.get('/trips'); }
   catch {
       const cached = await localCache.get('trips', Infinity);
       if (cached) set({ trips: cached });
   }
   ```

2. `syncQueue` — a persistent mutation queue backed by AsyncStorage:

   ```
   export const syncQueue = {
     async add(operation) {
       const queue = await this.getQueue();
       queue.push({ ...operation, timestamp: Date.now(), id });
       await AsyncStorage.setItem(QUEUE_KEY, JSON.stringify(queue));
     },
     async flush() {
       const queue = await this.getQueue();
       for (const op of sorted) {
         await api({ method: op.method, url: op.url, data: op.data });
       }
     }
   };
   ```

   When a write operation fails (network error), the store catches the exception and enqueues the mutation. The queue is flushed on reconnect via the Profile screen's "Sync Offline Changes" button, replaying operations in timestamp order.

Consequences

Positive:

- **Zero-downtime reads:** Users can browse all previously loaded trips, stops, and budget data without connectivity. The `localCache` serves stale data with infinite TTL, preferable to a blank screen.
- **No data loss:** Write operations are persisted to AsyncStorage immediately. Even if the app is closed and reopened, the queue survives and can be flushed.
- **Simple conflict resolution:** Mutations are replayed in timestamp order (FIFO). Combined with the server-side last-write-wins semantics (`updatedAt` timestamps), this provides eventual consistency without complex merge logic.
- **AsyncStorage is already a dependency:** Used for Firebase auth persistence (`getReactNativePersistence(AsyncStorage)` in `config/firebase.ts`), so no additional dependency is introduced.

Negative:

- **Stale data risk:** Offline users see the last-cached state, which may diverge significantly from the current server state if collaborators have been making changes.
- **Queue conflicts:** If two offline users make conflicting edits (e.g., both remove the same stop), the second flush will encounter a 404 or produce an inconsistent state. Last-write-wins resolves this but may silently discard one user's changes.
- **Manual sync trigger:** The queue does not auto-flush on reconnect in the current implementation; users must navigate to the Profile screen and tap "Sync Offline Changes." This is a usability gap acknowledged for future improvement.
- **AsyncStorage performance:** Large trip documents serialised to JSON may incur noticeable read/write latency; AsyncStorage is not designed for high-frequency updates.

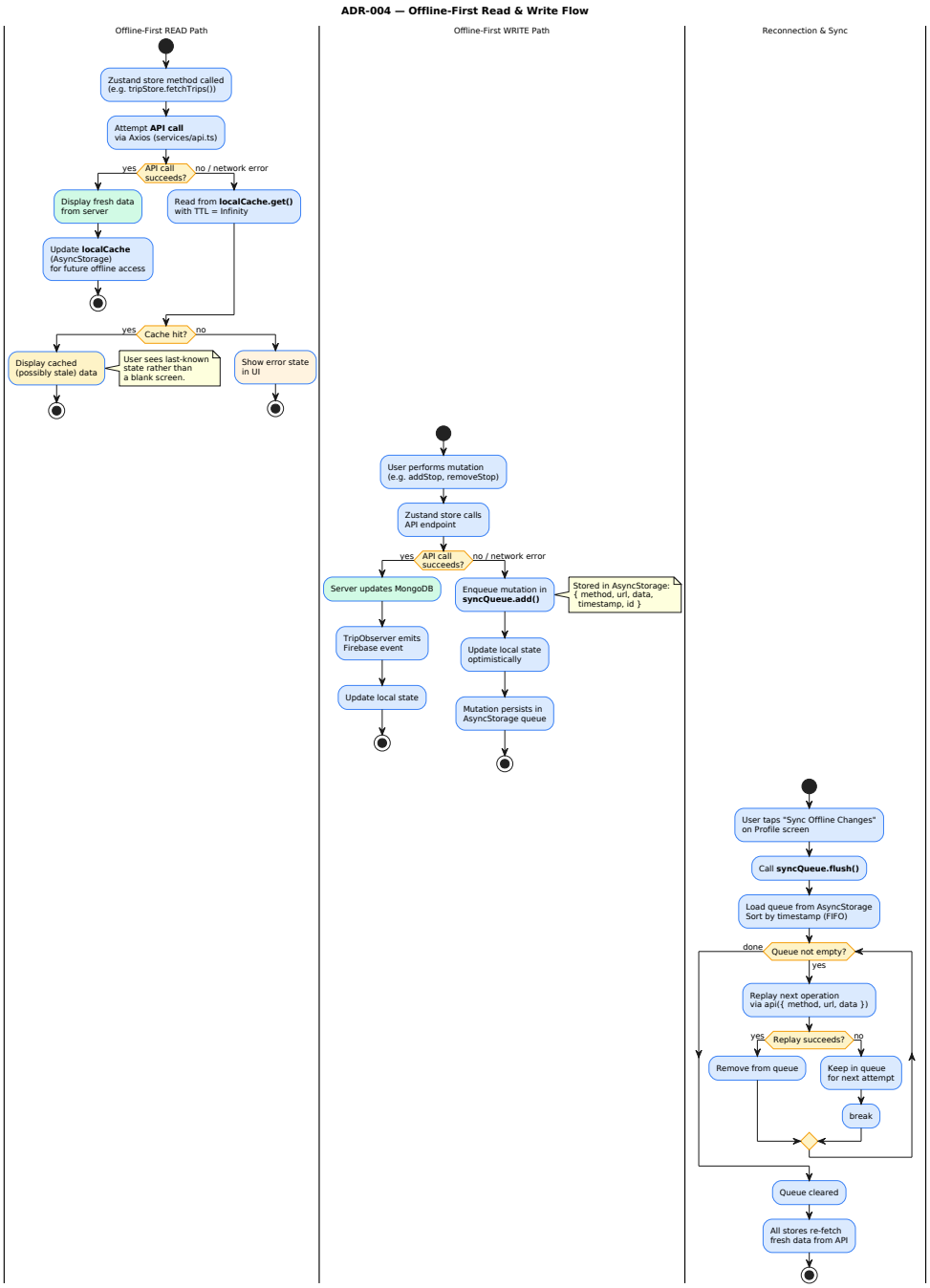

Figure 8: ADR-004 Activity Diagram: offline-first read and write flows. **Read path:** API call attempted → on failure, `localCache` serves stale data with TTL = Infinity. **Write path:** API call attempted → on failure, mutation enqueued via `syncQueue.add()` and persisted in AsyncStorage. **Sync path:** on reconnect, `syncQueue.flush()` replays operations in timestamp order.

4. ADR Summary and Cross-references

The following table summarises all four ADRs and maps them to the stakeholder concerns and NFRs they address:

ADR	Decision	Status	Concerns	NFRs Addressed
001	Firebase RTDB as thin event bus, MongoDB as source of truth	**Accepted**	C7 (Real-time), C3 (Offline)	NFR-03 (sync <500ms), FR-20/21 (collaboration)
002	API Gateway with secret proxying	**Accepte**	C2 (Security), C6 (Quota)	NFR-11 (no client secrets), NFR-06 (quota), NFR-10 (HTTPS)
003	Facade + Strategy for multi-provider POI	**Accepted**	C4 (Modifiability), C6 (Quota)	NFR-15 (swappability), NFR-08 (fallback), NFR-06 (caching)
004	Offline-first architecture with AsyncStorage	**Accepte**	C3 (Offline), C1 (Usability)	NFR-07 (offline availability), FR-03 (local persistence)

> **Inter-ADR dependencies.** ADR-001 (Firebase as event bus) depends on ADR-002 (API Gateway) because the gateway is the component that writes events to Firebase after persisting to MongoDB. ADR-003 (Facade pattern) is implemented within the gateway defined by ADR-002. ADR-004 (offline-first) complements ADR-001 by providing a client-side persistence layer that bridges connectivity gaps.

5. Conclusion

This document has presented the Architecture Framework for WanderMate following two complementary approaches:

IEEE 42010 Stakeholder Analysis identified five stakeholder classes (End Users, Development Team, Project Supervisor, System Administrator, API Providers) and seven concrete concerns spanning usability, security, offline availability, modifiability, deployment, quota compliance, and real-time collaboration. Four architecture viewpoints (Functional, Information, Deployment, Development) were defined with concrete views grounded in the codebase, providing full traceability from stakeholder concerns through viewpoints to specific code artefacts and requirements.

Nygard ADR documentation captured four major design decisions:

1. Using Firebase RTDB as a thin notification layer while MongoDB remains the authoritative data store (ADR-001).
2. Adopting the API Gateway pattern to protect secrets and centralise policy enforcement (ADR-002).
3. Implementing the Facade + Strategy pattern for multi-provider POI access with caching and fallback (ADR-003).
4. Building an offline-first client with AsyncStorage-backed caching and a persistent

sync queue (ADR-004).

Each decision was documented with its full context, the alternatives considered, the chosen approach with supporting code evidence, and an honest assessment of both positive and negative consequences. Together, these decisions form a coherent architectural strategy that balances the competing forces of real-time collaboration, offline availability, API quota constraints, security, and long-term maintainability.